

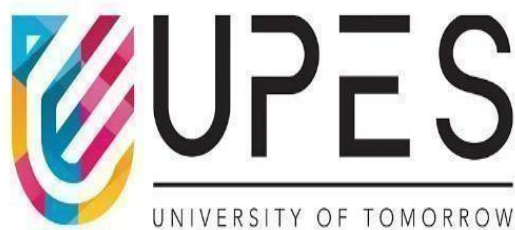
A

LABORATORY FILE

On
Applied Machine Learning

(CSAI2017P_5)

School of Computer Science, UPES, Dehradun.



B.TECH. – IV Semester

Jan. – May (2026)

Submitted to:

Dr.Sahinur Rahman Laskar

Assistant Professor

SOCS, UPES

Submitted by:

Name: Yash kangra

Batch: 19

Sap ID 590019256

MINI PROJECT REPORT

Performance Analysis of Machine Learning Models Using L1 and L2 Regularization for Regression and Classification Tasks

MINI PROJECT

Job Salary Prediction using Linear Regression, Lasso (L1) and Ridge (L2)

1. Problem Definition

Objective

The objective of this project is to analyze and compare the performance of machine learning regression models for predicting job salaries using professional and educational attributes.

This project studies the impact of:

- Linear Regression as baseline model
- Lasso Regression (L1 Regularization)
- Ridge Regression (L2 Regularization)

The goal is to determine which model gives better prediction accuracy while controlling overfitting.

Task Defined

Regression Task: Predict the continuous target variable **Salary** using job and employee related input features.

2. Data Understanding

Dataset Overview

The dataset used is: `job_salary_prediction_dataset.csv`

It contains employee/job records with salary as the target variable.

Dataset Features

Numerical Features:

- `experience_years`
- `skills_count`
- `certifications`

Categorical Features:

- `job_title`
- `education_level`
- `industry`
- `company_size`
- `location`
- `remote_work`

Target Variable: `salary`

3. Data Preprocessing

Duplicate Removal

```
df =  
df.drop_duplicates()
```

Handling Missing Values

Check if any data is missing; fill or remove missing values.

Encoding

Categorical columns converted using: `OneHotEncoder()`

Scaling

Numerical columns standardized using: `StandardScaler()`

Log Transformation

Target variable transformed using: `y = np.log1p(y)`

This reduces skewness and improves regression performance.

Train-Test Split

The dataset is divided into two parts:

- **Training Data (80%)** → used to train the model
- **Testing Data (20%)** → used to evaluate performance

```
train_test_split(test_size=0.2, random_state=42)
```

4. Model Training

Regression Models

Model	Type
Linear Regression	Normal Regression
Lasso Regression	L1 Regularization
Ridge Regression	L2 Regularization

Classification Models

Model	Type
Logistic Regression	Linear Classifier
Decision Tree	Tree Based
Random Forest	Ensemble

5. Model Training

Regression vs Classification Models

Linear Regression: Predicts continuous salary values using a linear relationship between input features and salary.

Lasso Regression (L1): Adds penalty equal to sum of absolute coefficients. Shrinks some coefficients to zero → performs feature selection.

Ridge Regression (L2): Adds penalty equal to square of coefficients. Reduces coefficient magnitude smoothly while retaining all features.

Logistic Regression: Used for classification. Predicts salary category probabilities (Low / Medium / High). **Decision Tree:** Splits data into rule-based branches to classify salary categories.

Random Forest: Combines multiple decision trees and averages predictions for better accuracy and lower overfitting.

Hyperparameters Used

Model	Parameter
Lasso Regression	alpha = 0.01
Ridge Regression	alpha = 1.0
Logistic Regression	max_iter = 1000

Decision Tree	max_depth = 6
Random Forest	n_estimators = 100

6. Prediction & Results

Regression Results

All regression models were trained on the training dataset and evaluated on the held-out test set. The target variable (salary) was log-transformed using `log1p()`, and predictions were converted back to original scale using `expm1()`.

Evaluation Metrics: R^2 Score, RMSE, MAE

Model	Test R^2	Test RMSE	Test MAE
Linear Regression	0.975335	5854.94	4611.58
Lasso Regression (L1)	0.724958	19551.75	15357.33
Ridge Regression (L2)	0.975337	5854.73	4611.43

- Ridge Regression achieved the best performance with the highest R^2 and lowest RMSE.
- Linear Regression performed almost identically to Ridge.
- Lasso Regression performed significantly worse, indicating over-regularization and loss of important features.

Classification Results

Salary values were converted into 3 categories using quantile-based binning (`qcut`): **Low**, **Medium**, **High**.

All classification models were evaluated using macro-averaging.

Model	Accuracy	Precision	Recall	F1 Score
Logistic Regression	0.91934	0.91973	0.91934	0.91950
Decision Tree	0.66466	0.67802	0.66466	0.66927
Random Forest	0.89090	0.89179	0.89090	0.89126

- Logistic Regression achieved the best performance across all metrics.
- Random Forest performed strongly, but slightly below Logistic Regression.
- Decision Tree performed the worst, indicating overfitting and poor generalization.

Overall Performance Summary

Rank	Regression Model	Performance	Classification Model	Performance
1	Ridge Regression	Best	Logistic Regression	Best
2	Linear Regression	Very Close	Random Forest	Strong
3	Lasso Regression	Poor	Decision Tree	Weak

7. Evaluation

Regression

- Ridge Regression (L2) and Linear Regression (OLS) performed almost identically, achieving veryhigh R^2 (~0.975) and low RMSE (~5854). This indicates that most features contribute meaningfully and do not require elimination.
- Lasso Regression (L1) showed significantly worse performance ($R^2 \approx 0.72$), with much higher RMSE and MAE. This suggests that L1 regularization removed important features, leading to underfitting.
- L2 is clearly superior in this dataset. L1 is too aggressive for feature selection.
- When most features are useful → Ridge is preferred. When many irrelevant features exist → Lasso is useful.

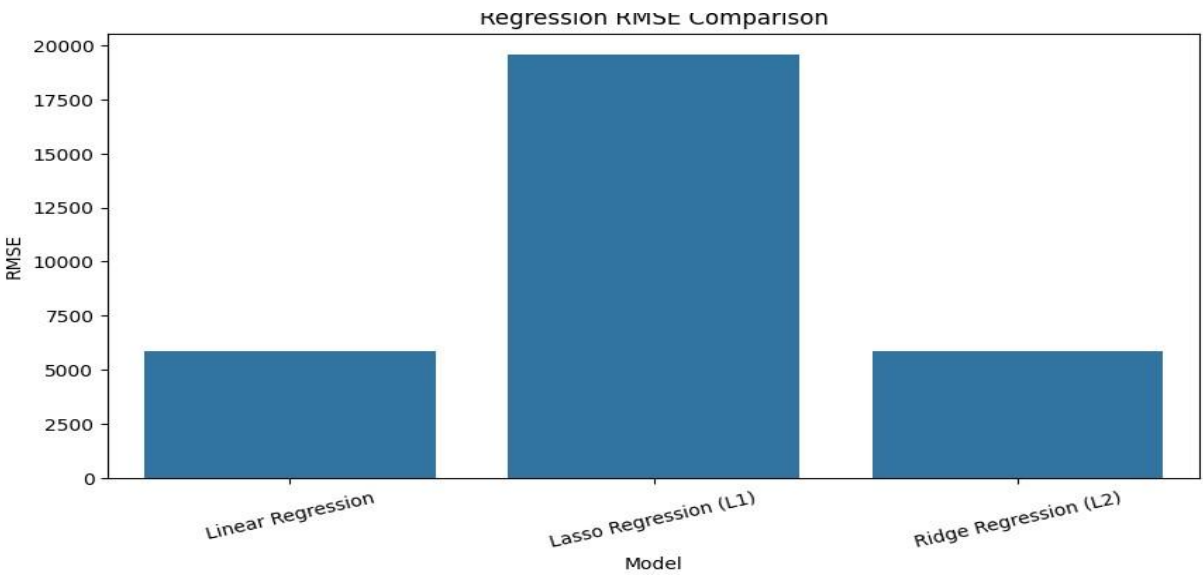
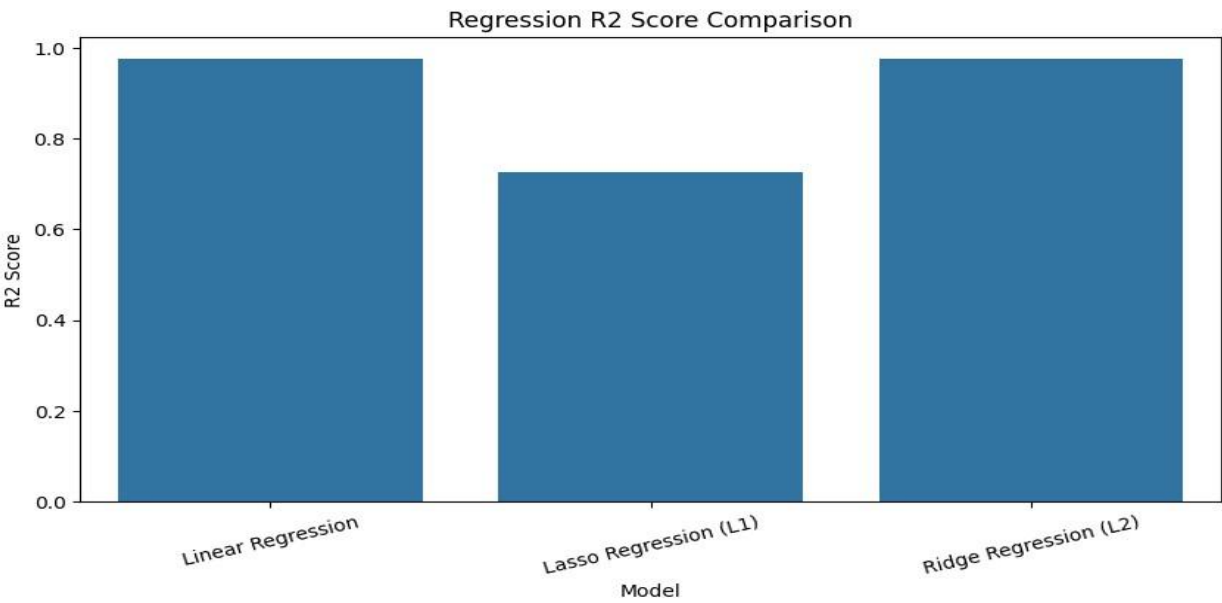
Classification

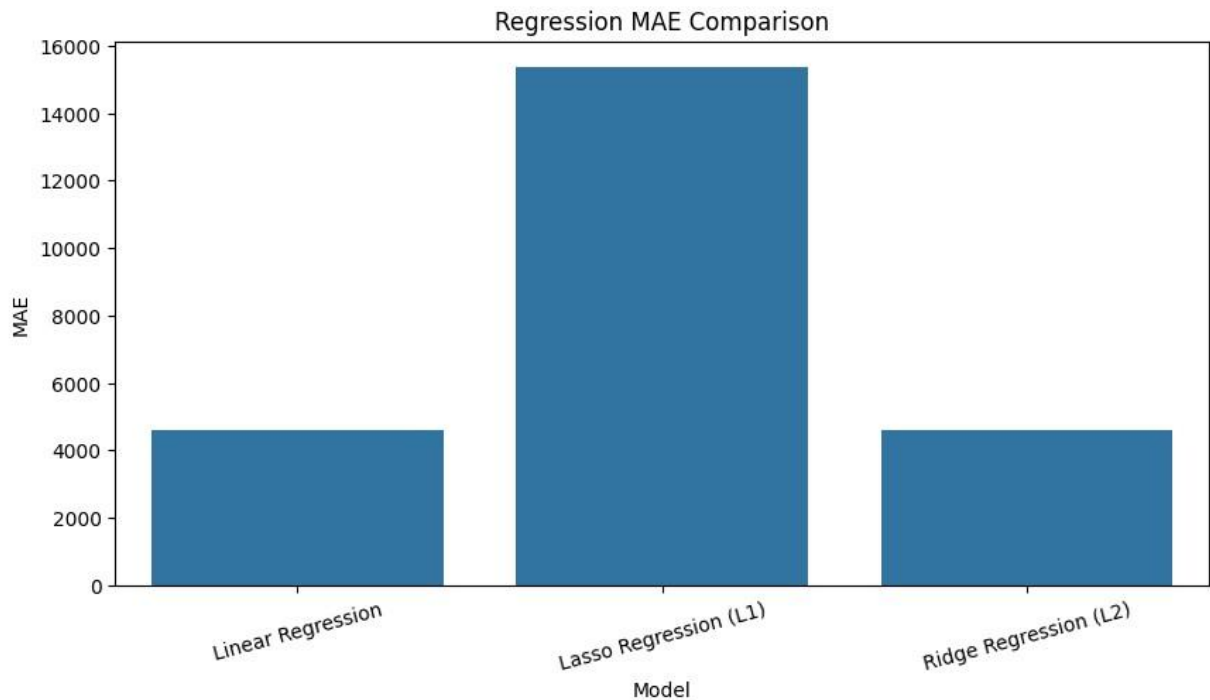
- Logistic Regression achieved the highest performance: Accuracy ≈ 0.919 , F1 Score ≈ 0.919 . Indicates strong linear separability of salary classes after preprocessing.
- Random Forest performed strongly (F1 Score ≈ 0.891). Captured non-linear patterns better than Decision Tree but slightly below Logistic Regression.
- Decision Tree performed poorly (Accuracy ≈ 0.66). Indicates overfitting and poor generalization on test data.
- Ensemble model (Random Forest) did not outperform linear model. Suggests dataset relationshipsare mostly linear.

L1 vs L2 Summary

Criterion	Linear	L1 (Lasso)	L2 (Ridge)
Feature Selection	No	Yes (zeros)	No
Multicollinearity Handling	Poor	Moderate	Excellent
Overfitting Control	None	Strong	Strong
Interpretability	Moderate	High (sparse)	Moderate

Performance in this Project	High	Low	Highest
Best Use Case	Simple relationships	Few important features	All features useful





Conclusion

- The project successfully implemented a **machine learning pipeline** integrating data preprocessing and model training in a single workflow.
- The use of **Pipeline and ColumnTransformer** ensured that:
 - Numerical features were scaled using StandardScaler
 - Categorical features were encoded using OneHotEncoder
 - Missing values were handled systematically
- This approach helped in:
 - Preventing data leakage
 - Maintaining consistency between training and testing data
 - Improving code efficiency and reproducibility
- In regression tasks, **Ridge Regression (L2)** performed best, indicating that all features contributed meaningfully and required regularization rather than elimination.
- In classification tasks, **Logistic Regression** achieved the highest accuracy, showing that the dataset is largely linearly separable.
- Overall, the pipeline-based approach improved:
 - Model performance
 - Scalability
 - Reliability of predictions


```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer

# Regression Models
from sklearn.linear_model import LinearRegression, Lasso, Ridge

# Classification Models
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

# Metrics
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score

from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay
```

```
file = "/content/job_salary_prediction_dataset.csv"
df = pd.read_csv(file)
df.head()
```

	job_title	experience_years	education_level	skills_count	industry	company_size	location	remote_work	certifications	salary
0	AI Engineer	10	Bachelor	2	Healthcare	Medium	India	Hybrid	2	109413
1	Data Analyst	5	Bachelor	17	Telecom	Small	Australia	No	0	93764
2	Frontend Developer	18	PhD	4	Media	Medium	Singapore	No	1	148123
3	Business Analyst	19	PhD	13	Retail	Medium	Canada	Yes	0	189123
4	Product Manager	15	Bachelor	7	Manufacturing	Large	Sweden	Yes	0	165069

```
print("Original Shape:", df.shape)

print("\nMissing Values:")
print(df.isnull().sum())

df = df.drop_duplicates()

print("\nShape after removing duplicates:", df.shape)
```

```
Original Shape: (250000, 10)

Missing Values:
job_title      0
experience_years 0
education_level 0
skills_count    0
industry        0
company_size    0
location        0
remote_work     0
certifications  0
salary         0
dtype: int64

Shape after removing duplicates: (250000, 10)
```

```
X = df.drop('salary', axis=1)
y = df['salary']

cat_cols = X.select_dtypes(include='object').columns
num_cols = X.select_dtypes(exclude='object').columns

print("Categorical Columns:", list(cat_cols))
print("Numerical Columns:", list(num_cols))

Categorical Columns: ['job_title', 'education_level', 'industry', 'company_size', 'location', 'remote_work']
Numerical Columns: ['experience_years', 'skills_count', 'certifications']
```

```
numeric_transformer = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

categorical_transformer = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(handle_unknown='ignore'))
])

preprocessor = ColumnTransformer([
    ('num', numeric_transformer, num_cols),
    ('cat', categorical_transformer, cat_cols)
])
```

```
y_reg = np.log1p(y)

X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
    X, y_reg,
    test_size=0.2,
    random_state=42
)
```

```
reg_models = {
    'Linear Regression': LinearRegression(),
    'Lasso Regression (L1)': Lasso(alpha=0.01),
    'Ridge Regression (L2)': Ridge(alpha=1.0)
}

reg_results = []
```

```
for name, model in reg_models.items():

    pipe = Pipeline([
        ('prep', preprocessor),
        ('model', model)
    ])

    pipe.fit(X_train_reg, y_train_reg)

    pred_log = pipe.predict(X_test_reg)

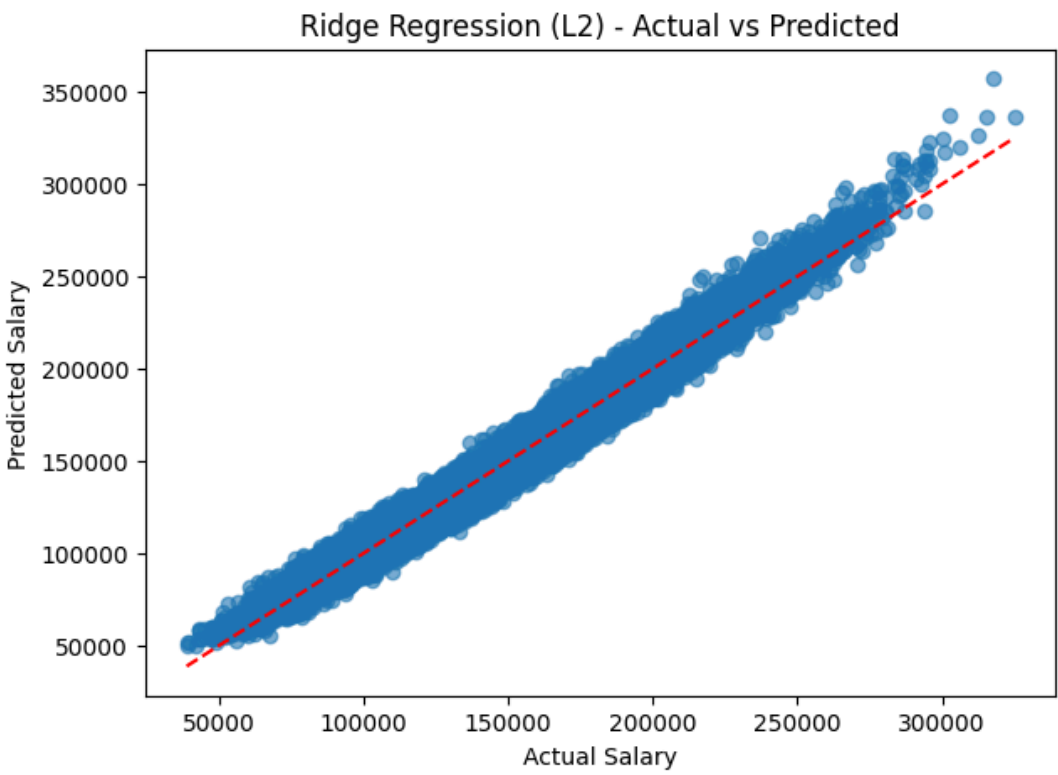
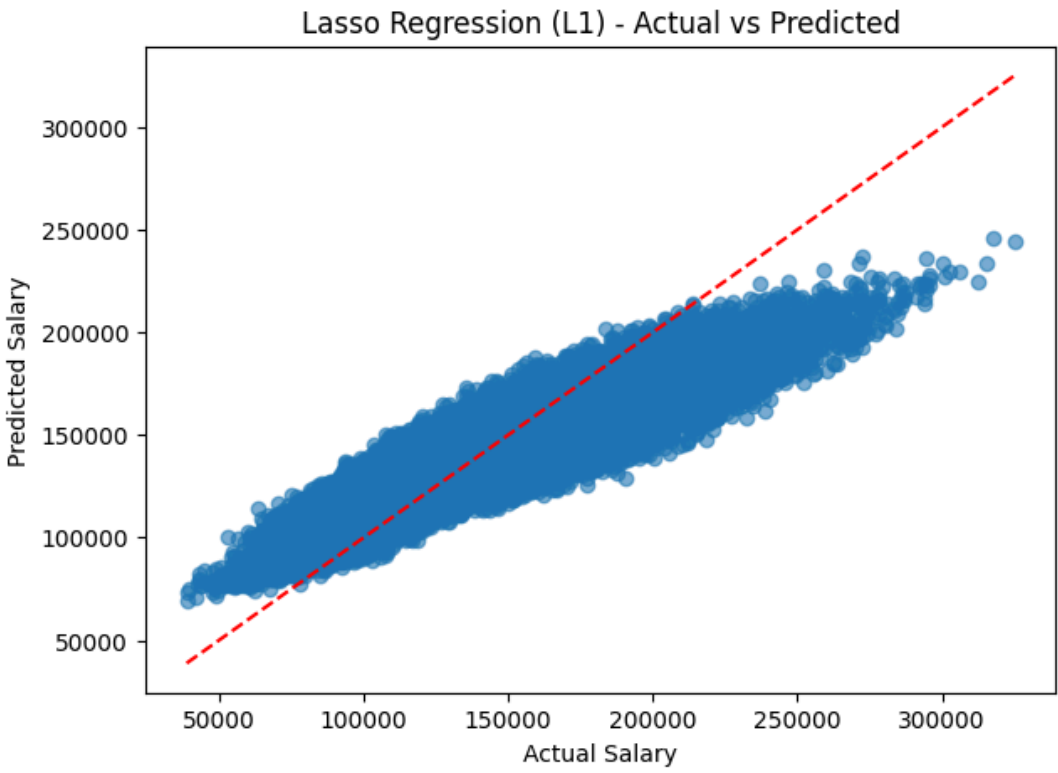
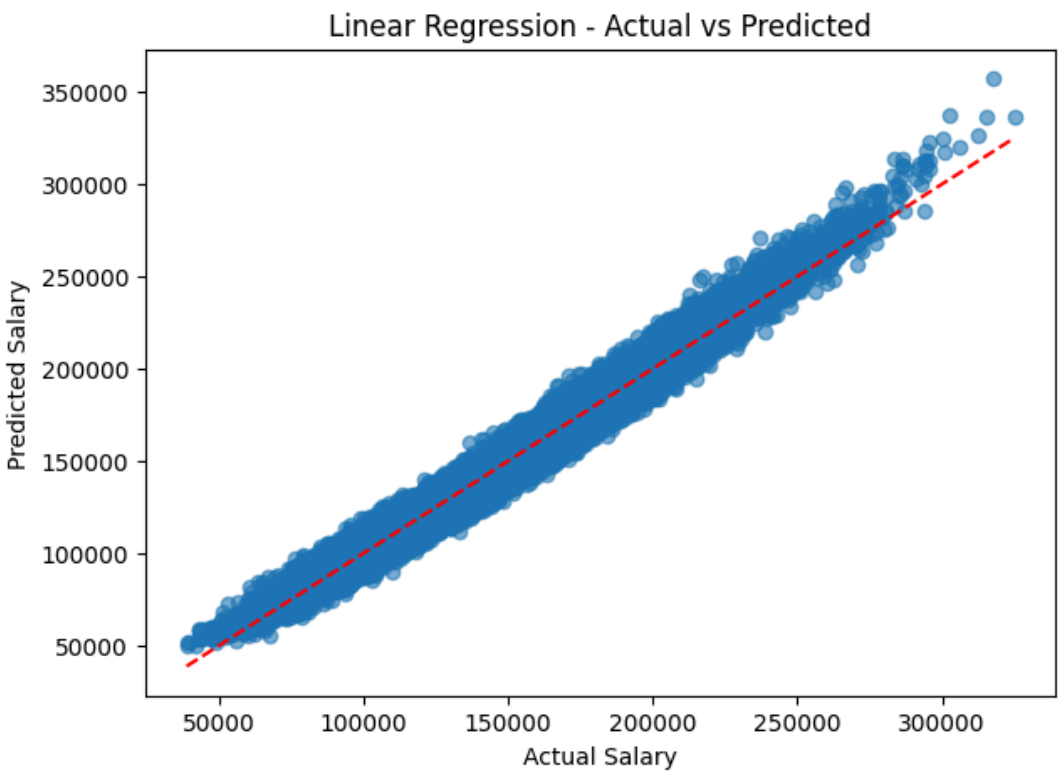
    pred = np.expm1(pred_log)
    actual = np.expm1(y_test_reg)

    mae = mean_absolute_error(actual, pred)
    rmse = np.sqrt(mean_squared_error(actual, pred))
```

```
r2 = r2_score(actual, pred)

reg_results.append([name, mae, rmse, r2])

plt.figure(figsize=(7,5))
plt.scatter(actual, pred, alpha=0.6)
plt.plot(
    [actual.min(), actual.max()],
    [actual.min(), actual.max()],
    'r--'
)
plt.xlabel("Actual Salary")
plt.ylabel("Predicted Salary")
plt.title(f"{name} - Actual vs Predicted")
plt.show()
```



```
reg_df = pd.DataFrame(
    reg_results,
    columns=['Model', 'MAE', 'RMSE', 'R2 Score']
)
```

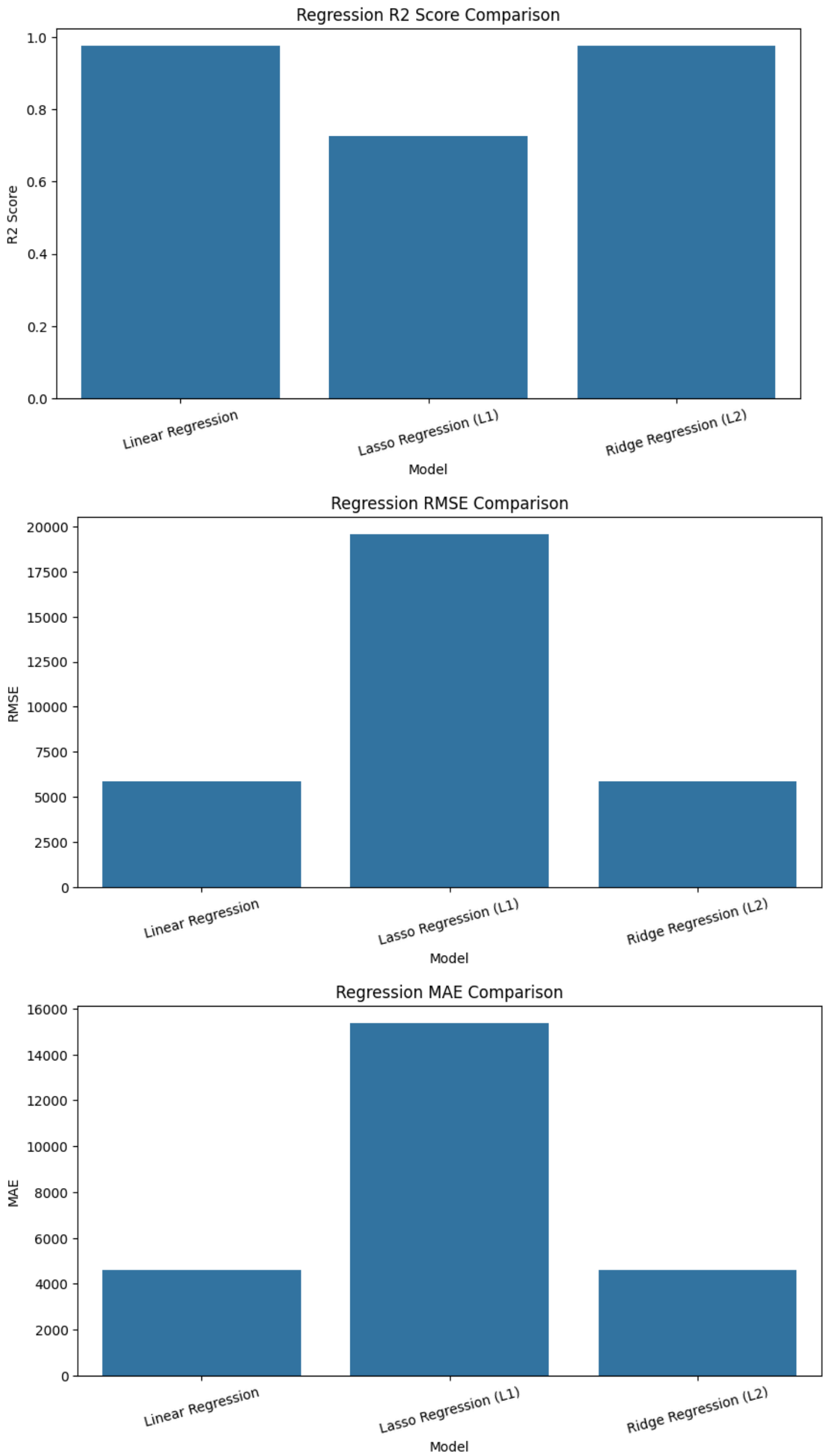
reg_df

	Model	MAE	RMSE	R2 Score
0	Linear Regression	4611.578436	5854.943845	0.975335
1	Lasso Regression (L1)	15357.334938	19551.754927	0.724958
2	Ridge Regression (L2)	4611.429922	5854.733855	0.975337

```
plt.figure(figsize=(10,5))
sns.barplot(x='Model', y='R2 Score', data=reg_df)
plt.title("Regression R2 Score Comparison")
plt.xticks(rotation=15)
plt.show()
```

```
plt.figure(figsize=(10,5))
sns.barplot(x='Model', y='RMSE', data=reg_df)
plt.title("Regression RMSE Comparison")
plt.xticks(rotation=15)
plt.show()
```

```
plt.figure(figsize=(10,5))
sns.barplot(x='Model', y='MAE', data=reg_df)
plt.title("Regression MAE Comparison")
plt.xticks(rotation=15)
plt.show()
```



```
df['salary_category'] = pd.qcut(
    df['salary'],
    q=3,
    labels=['Low', 'Medium', 'High']
)

df[['salary', 'salary_category']].head()
```

	salary	salary_category
0	109413	Low
1	93764	Low
2	148123	Medium
3	189123	High
4	165069	High

```
X_cls = df.drop(['salary', 'salary_category'], axis=1)
y_cls = df['salary_category']

X_train_cls, X_test_cls, y_train_cls, y_test_cls = train_test_split(
    X_cls, y_cls,
    test_size=0.2,
    random_state=42,
    stratify=y_cls
)
```

```
cls_results = []

for name, model in cls_models.items():

    pipe = Pipeline([
        ('prep', preprocessor),
        ('model', model)
    ])

    pipe.fit(X_train_cls, y_train_cls)

    pred = pipe.predict(X_test_cls)

    acc = accuracy_score(y_test_cls, pred)
    prec = precision_score(y_test_cls, pred, average='macro', zero_division=0)
    rec = recall_score(y_test_cls, pred, average='macro', zero_division=0)
    f1 = f1_score(y_test_cls, pred, average='macro', zero_division=0)

    cls_results.append([name, acc, prec, rec, f1])

print("\n", name)
print("Accuracy :", acc)
print("Precision :", prec)
```



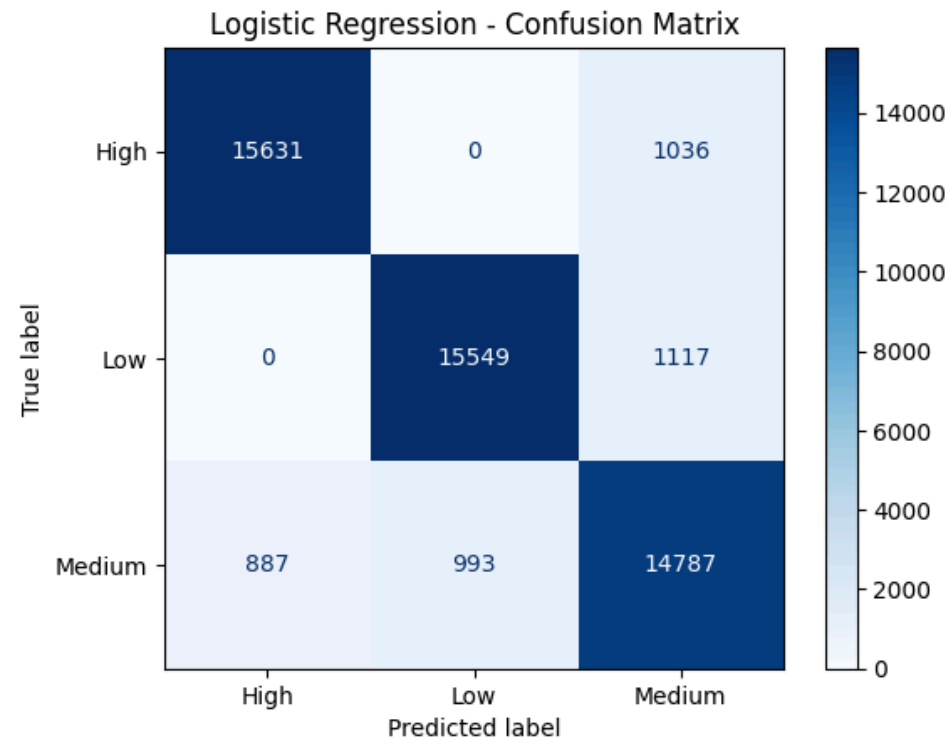
```
print("Recall   :", rec)
print("F1 Score :", f1)

cm = confusion_matrix(y_test_cls, pred)

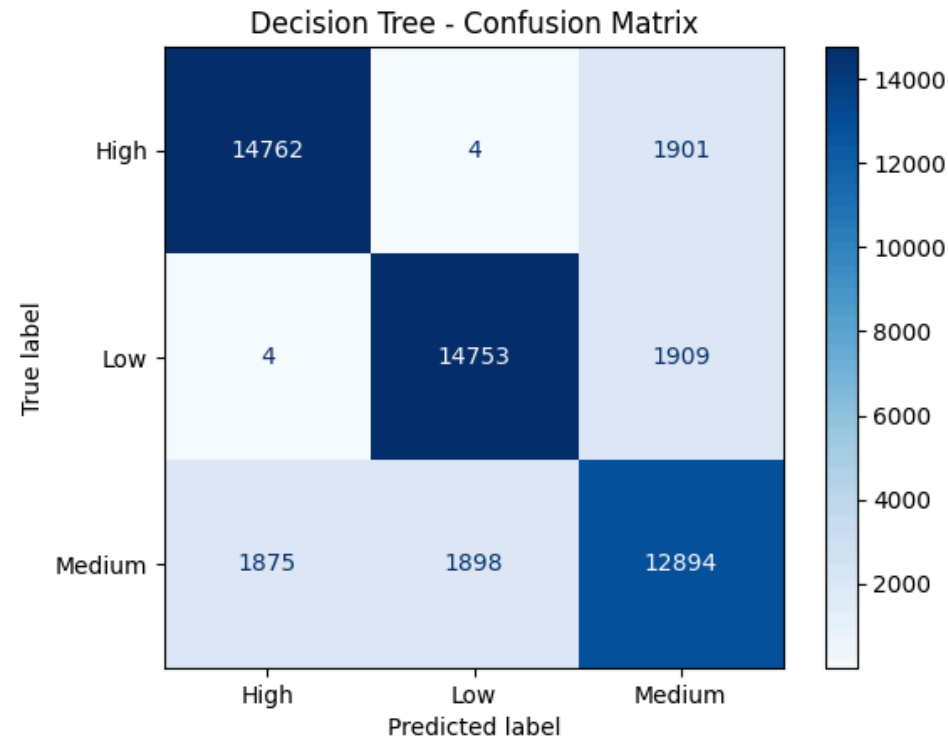
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=np.unique(y_test_cls)
)

disp.plot(cmap='Blues')
plt.title(F"{name} - Confusion Matrix")
plt.show()
```

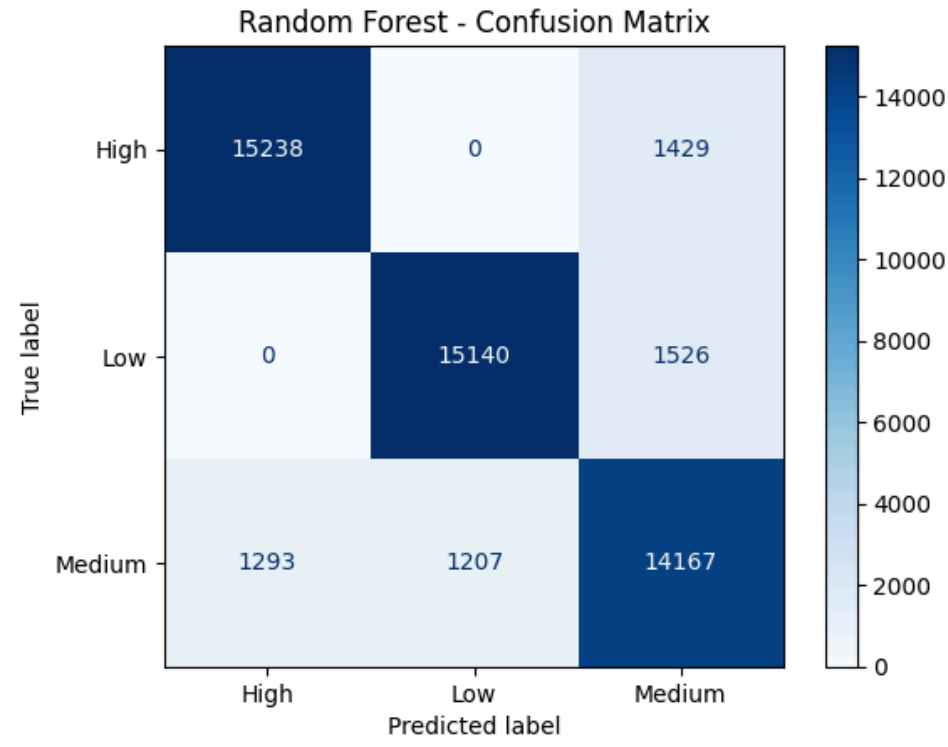
Logistic Regression
Accuracy : 0.91934
Precision : 0.9197254520918933
Recall : 0.9193402727409271
F1 Score : 0.919502821595599



Decision Tree
Accuracy : 0.84818
Precision : 0.8482656973985613
Recall : 0.8481807406933584
F1 Score : 0.8482226901842888



Random Forest
Accuracy : 0.8909
Precision : 0.8917873866371583
Recall : 0.8909003507197347
F1 Score : 0.8912594887837679



```
cls_df = pd.DataFrame(
    cls_results,
    columns=['Model', 'Accuracy', 'Precision', 'Recall', 'F1 Score']
)
```

cls_df

	Model	Accuracy	Precision	Recall	F1 Score
0	Logistic Regression	0.91934	0.919725	0.919340	0.919503
1	Decision Tree	0.84818	0.848266	0.848181	0.848223
2	Random Forest	0.89090	0.891787	0.890900	0.891259

```
plt.figure(figsize=(10,5))
sns.barplot(x='Model', y='Accuracy', data=cls_df)
plt.title("Accuracy Comparison")
plt.xticks(rotation=15)
plt.show()
```

```
plt.figure(figsize=(10,5))
sns.barplot(x='Model', y='Precision', data=cls_df)
plt.title("Precision Comparison")
plt.xticks(rotation=15)
plt.show()
```

```
plt.figure(figsize=(10,5))
sns.barplot(x='Model', y='Recall', data=cls_df)
plt.title("Recall Comparison")
plt.xticks(rotation=15)
plt.show()
```